# Отказоустойчивые системы

## Лекция 1. Введение: отказы, метрики, принципы

IT-специальность, 4 курс · 90 минут

# План лекции

| Время, мин | Блок | Слайды |
|---|---|---|
| 0–10 | Мотивация: цена отказа | 4–5 |
| 10–25 | Терминология: дефект, ошибка, отказ; dependability | 6–9 |
| 25–40 | Классификация отказов | 10–13 |
| 40–50 | Перерыв | — |
| 50–65 | Метрики: MTTF, MTTR, MTBF, доступность, SLO | 14–19 |
| 65–80 | Принципы построения: избыточность, этапы, SPOF | 20–24 |
| 80–90 | Задачи, итоги, домашнее задание | 25–29 |

# Что вы будете уметь после лекции

- Отличать **дефект**, **ошибку** и **отказ** и строить цепочку между ними
- Классифицировать отказы по длительности, поведению и природе
- Считать **доступность** последовательных, параллельных и смешанных схем
- Называть принципы и этапы построения отказоустойчивых систем
- Объяснять, почему избыточность работает только при **независимых** отказах

# Цена отказа (1/2)

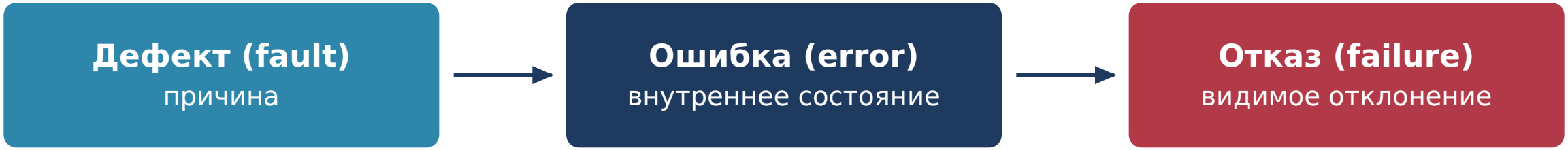
| Кейс | Что произошло | Урок |
|---|---|---|
| **Ariane 5**, рейс 501<br>4 июня 1996<br>≈ $370 млн | Через ≈37 с после старта ракета самоуничтожилась. В навигации, взятой из Ariane 4, 64-битное число с плавающей точкой не поместилось в 16-битное целое. Основной и резервный блоки выполняли **один и тот же код** и отказали одинаково. | Копии одного кода не спасают от ошибки проектирования. Повторное использование кода требует проверки допущений. |
| **Therac-25**<br>1985–1987 | Аппарат лучевой терапии. Состояние гонки в ПО и отказ от аппаратных блокировок привели к 6 случаям тяжёлой передозировки, минимум трое пациентов погибли. | В системах, где отказ стоит жизни, нужны независимые аппаратные защиты. |

# Цена отказа (2/2)

| Кейс | Что произошло | Урок |
|---|---|---|
| **Knight Capital**<br>1 августа 2012<br>≈ $440 млн | Новую версию ПО развернули на 7 из 8 серверов. На восьмом повторно использованный флаг включил старый, давно не используемый код. За 45 минут — огромный поток ошибочных сделок. | Развёртывание — тоже источник отказов. Нужны автоматизация деплоя и аварийный выключатель. |
| **AWS S3**, us-east-1<br>28 февраля 2017<br>≈ 4 часа | Оператор ввёл команду с неверным параметром и вывел из работы больше серверов, чем планировал. Пострадали тысячи сайтов и сервисов. | Система должна ограничивать масштаб ручных действий. |
| **CrowdStrike**<br>19 июля 2024 | Ошибочное обновление агента безопасности вызвало «синий экран» (BSOD) примерно на 8,5 млн компьютеров с Windows. | Поэтапное развёртывание (staged rollout, canary). |

**Вопрос:** что общего у Ariane 5 и CrowdStrike?

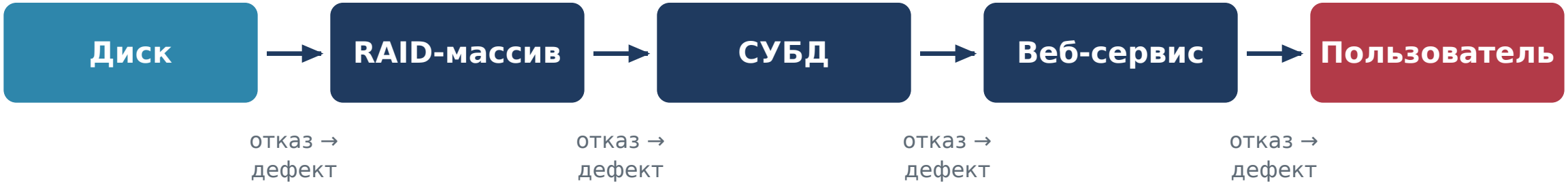
# Дефект → Ошибка → Отказ




- **Дефект (fault)** — причина: баг, сбой железа, неверная конфигурация, действие оператора
- **Ошибка (error)** — неверное внутреннее состояние, которое может привести к отказу
- **Отказ (failure)** — наблюдаемое снаружи отклонение поведения от спецификации

По таксономии Avizienis, Laprie, Randell, Landwehr (IEEE TDSC, 2004)

# Цепочка рекурсивна




- Отказ компонента — это **дефект** для системы уровнем выше
- Граница между терминами зависит от уровня рассмотрения
- Дефект может «спать» годами; ошибка может быть **замаскирована** (ECC исправит бит)

# Dependability: атрибуты

| Атрибут | Смысл | Вопрос |
|---|---|---|
| **Reliability** — безотказность | Непрерывность корректной работы | Как долго проработает без отказа? |
| **Availability** — доступность | Готовность в нужный момент | Работает ли прямо сейчас? |
| **Safety** — безопасность | Отсутствие катастрофических последствий | Не навредит ли людям и среде? |
| **Integrity** — целостность | Отсутствие недопустимых изменений | Не испорчены ли данные? |
| **Maintainability** — ремонтопригодность | Лёгкость диагностики и ремонта | Как быстро починим? |

Dependability — способность предоставлять сервис, которому можно обоснованно доверять.

# Четыре способа достижения

| Подход | Что делаем | Примеры |
|---|---|---|
| **Fault prevention** — предотвращение | Не допускаем появления дефектов | Строгие требования, типизация, качественные компоненты |
| **Fault removal** — устранение | Находим и убираем дефекты | Тестирование, code review, верификация |
| **Fault forecasting** — прогнозирование | Оцениваем число и последствия дефектов | Модели надёжности, анализ рисков |
| **Fault tolerance** — терпимость | Работаем корректно несмотря на дефекты | Резервирование, повтор, откат ← **тема курса** |

Все дефекты устранить невозможно → проектируем так, чтобы дефекты не становились отказами.

# Отказы по длительности

| Тип | Описание | Пример | Защита |
|---|---|---|---|
| **Постоянные** (permanent) | Не исчезают сами | Сгоревший блок питания, вышедший из строя диск | Замена, резервирование |
| **Переходные** (transient) | Возникают и пропадают | Сбой бита от космической частицы, потеря сетевого пакета | Повтор (retry), ECC |
| **Перемежающиеся** (intermittent) | Возникают периодически из-за скрытой причины | Плохой контакт, перегрев, состояние гонки | Сложнее всего диагностировать |

Переходных отказов в реальных системах намного больше, чем постоянных.

# Отказы по поведению

| Класс | Поведение компонента |
|---|---|
| **Crash / fail-stop** | Остановился и молчит (fail-stop: остановка надёжно обнаруживается) |
| **Omission** — пропуск | Не отправил или не принял сообщение |
| **Timing** — временной | Ответ верный, но слишком поздно (или рано) |
| **Response / value** — по значению | Ответ пришёл вовремя, но неверный |
| **Arbitrary / Byzantine** — произвольный | Любое поведение, включая противоречивые ответы и злой умысел |

Сверху вниз: класс шире → защита дороже. Для $f$ отказов: crash — $f+1$ реплик (консенсус — $2f+1$), византийские — $3f+1$.

Классификация Ф. Кристиана (F. Cristian, 1991)

# Природа отказов. Bohrbug и Heisenbug

| Источник | Примеры |
|---|---|
| Аппаратные | Износ, брак, перегрев, излучение |
| Программные | Баги, утечки ресурсов, переполнения |
| Проектирования | Неверные требования и допущения (Ariane 5) |
| Эксплуатационные | Оператор, конфигурация, развёртывание (AWS, Knight) |
| Внешние | Питание, охлаждение, сеть провайдера, стихия |

- **Bohrbug** — детерминированный, воспроизводится, ловится тестами
- **Heisenbug** — зависит от тайминга и состояния; часто помогает перезапуск

# Что страшнее?

## Сервер **упал** — или **отвечает, но неверно**?

- **Silent data corruption** — тихая порча данных хуже падения
- Падение заметно сразу; порча расходится по системе и резервным копиям
- Поэтому контрольные суммы нужны на всех уровнях

| **На перерыв (10 мин)** |
| --- |
| Банкомат при нехватке купюр выдаёт сообщение и прекращает выдачу. Это отказ? С чьей точки зрения? |

# Метрики надёжности

| Метрика | Расшифровка | Смысл |
|---|---|---|
| **MTTF** | Mean Time To Failure | Среднее время до отказа |
| **MTTR** | Mean Time To Repair / Recover | Среднее время восстановления |
| **MTBF** | Mean Time Between Failures | Среднее время между отказами = MTTF + MTTR |
| **λ** (лямбда) | Failure rate | Интенсивность отказов, 1/ч |

- Экспоненциальная модель (λ = const): $\mathbf{R(t) = e^{-\lambda t}}$, MTTF = 1/λ
- Пример: λ = 0,001 1/ч → MTTF = 1000 ч; $R(100) = e^{-0,1} \approx 0,905$

# «Ванная кривая»




MTBF 1 000 000 ч ≠ «проживёт 114 лет»: это статистика по выборке в период нормальной эксплуатации.

# Доступность

$$A = MTTF / (MTTF + MTTR)$$

- Два пути роста: реже ломаться (MTTF ↑) или быстрее чиниться (MTTR ↓)
- Обычно дешевле сокращать **MTTR**: автоматический failover, быстрый откат, runbook
- MTTF = 1000 ч, MTTR = 2 ч → A = 1000/1002 ≈ **99,8%**
- Тот же MTTF, MTTR = 5 мин → A ≈ **99,99%**

# «Девятки» доступности

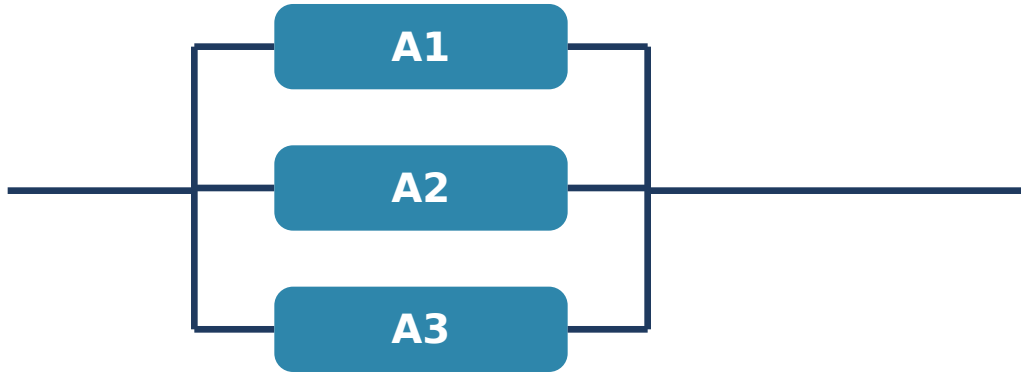
| Доступность | Простой в год | Простой в месяц (30 дн.) |
|---|---|---|
| 99% («две девятки») | ≈ 3,65 суток | ≈ 7,2 ч |
| 99,9% («три девятки») | ≈ 8,76 ч | ≈ 43 мин |
| 99,99% («четыре девятки») | ≈ 52,6 мин | ≈ 4,3 мин |
| 99,999% («пять девяток») | ≈ 5,26 мин | ≈ 26 с |

Простой 1 час раз в год или 1 минута каждую неделю — что лучше для пользователя?

# Последовательно и параллельно




**Последовательно: нужны все**




**Параллельно: достаточно одного**

- Последовательно: $\mathbf{A = A_1 \cdot A_2 \cdot \ldots \cdot A_n}$ — доступность падает с каждым звеном
- Параллельно: $\mathbf{A = 1 - (1 - A_1)(1 - A_2)\ldots(1 - A_n)}$ — перемножаются недоступности
- Параллельная формула верна только при **независимых** отказах!

# SLI, SLO, SLA и бюджет ошибок

| Термин | Расшифровка | Пример |
|---|---|---|
| **SLI** | Service Level Indicator — показатель | Доля успешных запросов; задержка p99 |
| **SLO** | Service Level Objective — цель | 99,9% успешных запросов за 30 дней |
| **SLA** | Service Level Agreement — договор | При доступности < 99,9% — компенсация клиенту |
| **Error budget** | Бюджет ошибок = 1 − SLO | ≈ 43 мин «плохого» времени в месяц |

Бюджет исчерпан → замораживаем релизы и занимаемся надёжностью.

# Виды избыточности

| Вид | Суть | Примеры |
|---|---|---|
| **Аппаратная** | Дополнительные компоненты | Два блока питания, RAID, резервные серверы |
| **Информационная** | Дополнительные данные для обнаружения и исправления ошибок | Бит чётности, код Хэмминга, CRC, ECC |
| **Временная** | Повторное выполнение | Повтор запроса, повторная передача пакета |
| **Программная** | Разные реализации одной функции | N-версионное программирование, блоки восстановления |

Избыточность без проверенного механизма переключения — иллюзия защиты.

# Как система обрабатывает отказ

| Этап | Механизмы |
|---|---|
| **1. Обнаружение** | Heartbeat, watchdog, контрольные суммы, assertions, health check, сравнение реплик |
| **2. Локализация и изоляция** | Найти неисправный компонент; bulkhead, зоны отказа, отключение узла |
| **3. Восстановление** | Backward: откат к checkpoint, транзакции · Forward: failover, коррекция кодом |
| **4. Устранение причины** | Ремонт, замена, исправление; постмортем |

**Маскирование** (TMR, ECC): без паузы, но дорого. **Обнаружение + восстановление**: дешевле, но с паузой.

# Принципы проектирования

- **Нет единой точки отказа** (SPOF — Single Point of Failure)
- **Независимость и разнообразие** реплик: разные стойки, питание, зоны, версии
- **Fail fast** — быстро и явно упасть лучше, чем работать с испорченным состоянием
- **Graceful degradation** — частичная потеря функций вместо полного отказа
- **Простота** — механизм защиты сам может стать причиной сбоя
- **Design for failure** — отказ норма; переключения нужно регулярно тестировать

# Найдите единые точки отказа




Какие компоненты — SPOF? Что ещё не нарисовано на схеме, но тоже может быть SPOF?

# Единые точки отказа: ответ




- На схеме: DNS-провайдер, балансировщик, БД, хранилище, единый ЦОД
- Вне схемы: сетевой канал, система деплоя, учётная запись администратора, один инженер, который «всё знает»

# Задачи

- **1.** 5 последовательных компонентов по 99,9%. Доступность системы и простой в год?
- **2.** Балансировщик (99,99%) → 2 параллельных app-сервера (по 99,5%) → БД (99,9%). Доступность? Что даст вторая БД (реплика, тоже 99,9%)?
- **3.** 3 сервера по 99% работают параллельно, но все подключены к одному коммутатору (99,9%). Реальная доступность?

Год = 8760 ч. Простой = $(1 - A) \times 8760$.

# Ответы: задачи 1 и 2

| № | Решение | Вывод |
|---|---|---|
| **1** | $0{,}999^5 \approx 0{,}9950 \rightarrow$ **99,5%**<br>Простой $\approx 0{,}005 \times 8760 \approx$ **43,7 ч/год** | 5 звеньев по 99,9% дают всего ≈ 99,5% |
| **2** | App: $1 - 0{,}005^2 = 0{,}999975$<br>$0{,}9999 \times 0{,}999975 \times 0{,}999 \approx$ **99,89%** (≈ 9,9 ч/год)<br>Две БД: $1 - 0{,}001^2 = 0{,}999999$<br>Итог ≈ **99,987%** (≈ 1,1 ч/год) | Резервировать **самое слабое звено**; после этого узким местом станет балансировщик |

# Ответ: задача 3

| Расчёт | Результат | Простой в год |
|---|---|---|
| 3 сервера параллельно: $1 - 0{,}01^3$ | 99,9999% | ≈ 31 с |
| × общий коммутатор 0,999 | ≈ **99,9%** | ≈ **8,8 ч** |

- Общая причина уничтожает выигрыш от резервирования
- Ariane 5, CrowdStrike: одна ошибка размножилась на все копии
- Считаем не только узлы, но и **общие зависимости**: питание, сеть, ПО, деплой, DNS

# Итоги

- Отказ неизбежен; цель — чтобы дефекты не становились отказами пользователя
- Дефект → ошибка → отказ; граница зависит от уровня рассмотрения
- Доступность зависит и от MTTF, и от MTTR; сокращать MTTR часто выгоднее
- Избыточность работает при независимых отказах и проверенном переключении
- Каждая «девятка» стоит денег — отказоустойчивость всегда компромисс

Какая система в вашей жизни обязана быть отказоустойчивой, а где защита не окупится?

# Домашнее задание и литература

- **Постмортем:** выбрать публичный разбор крупного сбоя (AWS, Google Cloud, Cloudflare, GitHub, GitLab, Atlassian) и на 1–2 страницах описать: цепочку дефект → ошибка → отказ, класс отказа, обнаружение и восстановление, что можно было сделать иначе
- **Задачи:** 3–4 расчёта доступности (последовательная, параллельная, смешанная схемы; MTTF/MTTR)

| Литература |
| --- |
| Avizienis A. et al. Basic Concepts and Taxonomy of Dependable and Secure Computing. IEEE TDSC, 2004 |
| Beyer B. et al. Site Reliability Engineering. Google, 2016 — главы об SLO и инцидентах |
| Kleppmann M. Designing Data-Intensive Applications, гл. 1 |
| Koren I., Krishna C. M. Fault-Tolerant Systems |